Security Audit: Docker-Based Web Application Deployment

Architecture and scan scope

For this task I deployed a small containerised web application using Docker Compose, made up of an Nginx front end container and a Flask backend container running on a local host. The two containers communicated over a shared Docker network, with the Nginx container exposing port 80 to the host and the Flask service kept internal wherever possible. This setup was chosen because it reflects a fairly common pattern for lightweight cloud applications, where a reverse proxy sits in front of an application server, and it gave me a realistic but manageable scope to audit rather than something overly complex. The scan scope covered the host machine, both running containers, and the exposed network ports, rather than any wider infrastructure.

Scanning method and configuration

I used Greenbone Community Edition, the open source version of OpenVAS, running it against the host IP address with the default full and fast scan configuration to begin with. Once the initial scan completed I ran a second, more targeted scan focused specifically on the exposed HTTP port, since that seemed like the area most likely to surface application level issues rather than just host level ones. Ioana et al. (2017) point out that serverless and containerised environments introduce a different security surface compared to traditional infrastructure, partly because of how quickly services can be spun up and torn down, and this was something I kept in mind when interpreting the scan results, since a container that looked fine at scan time might not reflect how the image behaves once redeployed.

Vulnerabilities identified

The scan flagged a handful of issues. The most significant was an outdated base image on the Flask container, which was still running an older Python image with several known CVEs relating to outdated system packages. It also picked up on the container running as root rather than a non root user, which is a common but risky default in Docker setups. Beyond that, there were a couple of medium severity findings around missing HTTP security headers on the Nginx container, specifically the absence of a Content Security Policy and X Frame Options header, and a low severity finding relating to verbose error messages being returned by the Flask application in debug mode.

Severity and impact

The outdated base image and the root user configuration were the two findings I would treat as high priority. Running a container as root means that if the

application itself is ever compromised, an attacker has a much easier path to escalating privileges on the host, which is a well documented risk in container security. The missing headers and verbose error output are lower severity individually, but they still widen the attack surface unnecessarily, particularly the debug mode issue, since it can leak internal application details that would otherwise help an attacker map out the system.

Recommended mitigations

To address these findings, I would rebuild the Flask image on a current, patched base image and rerun the scan to confirm the CVEs are resolved. Adding a dedicated non root user in the Dockerfile would deal with the privilege issue, and switching debug mode off before anything resembling production is straightforward but easy to forget. The missing headers can be added directly in the Nginx configuration file with minimal effort. None of these fixes are particularly time consuming, which made it a useful reminder that a lot of container security issues come down to defaults not being changed rather than genuinely hard problems.

Alignment with ISO/IEC 27001

Mapping these findings against ISO/IEC 27001 controls, the outdated image issue sits under vulnerability management and secure configuration, since it relates directly to keeping software patched and configured securely by default. The root user finding falls under access control, given that it concerns unnecessary privilege. The missing security headers and verbose error handling relate more to secure configuration and, to some extent, logging and monitoring, since better logging practices would help catch this kind of oversight earlier rather than relying solely on a periodic scan. Sharma and Kumar (2024) discuss how hybrid and multi cloud environments make consistent security governance harder to maintain across different deployment models, and this audit reinforced that idea for me, since even a small, single host deployment already had several gaps that would be easy to miss without a structured scan.

Limitations and future improvements

This audit was limited in a few ways. It only covered a single host and a small two container setup, so it does not capture the added complexity of a multi node or orchestrated environment such as Kubernetes. It also relied on a point in time scan, which does not account for configuration drift or vulnerabilities introduced after the scan was run. For future work, I think it would be worth integrating a scanning step directly into a CI/CD pipeline, so that images are checked automatically before deployment rather than audited after the fact, and combining OpenVAS with a container specific scanner such as Trivy would likely give a more complete

picture, since OpenVAS is generally stronger at host and network level scanning than at inspecting image layers directly.

References

Ioana, B. et al. (2017) *Survey of serverless computing: current trends and open problems*.

Sharma, S.K. and Kumar, P.B.S. (2024) 'Hybrid IT and multi cloud: an emerging trend and improved deployment model', *SN Computer Science*, 5(1), pp. 1 to 15.